UNISALES - SISTEMAS DE INFORMAÇÃO

Relatório Técnico
Documentação da DAG e Execução no Apache Airflow

Projeto de Análise e Fluxo de Dados - OLAP e ETL
AdventureWorks Data Warehouse

Atividade	Documentar a DAG e a execução no Airflow
DAG	etl_adventureworks_dw
Orquestrador	Apache Airflow 3.3.1
Executor	LocalExecutor
Data da validação	26/10/2026

Status: atividade concluída e validada com sucesso

1. Objetivo

Documentar a criação, validação e execução da DAG responsável por orquestrar a ETL incremental do Data Warehouse AdventureWorks no Apache Airflow. A atividade teve como foco comprovar que as dimensões são processadas antes da tabela fato, que a DAG é reconhecida sem erros de importação, que as tarefas são executadas com sucesso e que a execução não introduz duplicidades nem altera indevidamente os watermarks de controle incremental.

2. Contexto da orquestração

Antes da criação da DAG, os módulos de extração, transformação e carga já haviam sido validados individualmente. Também já estavam comprovados o carregamento das três dimensões e da tabela fato, a idempotência das cargas e o mecanismo incremental baseado em watermark. A etapa de Airflow, portanto, teve a função de integrar os componentes já testados em um fluxo único e observável.

Componente	Configuração utilizada
Apache Airflow	3.3.1
Executor	LocalExecutor
Banco de metadados	PostgreSQL 16
Banco do DW	PostgreSQL 16 - adventureworks_dw
Origem	SQL Server - AdventureWorks2016
Interface Web	localhost:8080

3. Definição da DAG

Foi criado o arquivo dags/etl_adventureworks_dw.py. A DAG foi definida com schedule=None para que, durante a validação, as execuções fossem disparadas manualmente. Também foram configurados catchup=False e as etiquetas adventureworks, etl, dw e incremental.

```
@dag(
    dag_id="etl_adventureworks_dw",
    schedule=None,
    start_date=datetime(2026, 1, 1),
    catchup=False,
    tags=["adventureworks", "etl", "dw", "incremental"],
)
```

Cada tarefa abre suas próprias conexões com o SQL Server e o PostgreSQL, chama a função incremental correspondente e fecha as conexões em bloco finally. Essa separação permite que o Airflow registre individualmente o sucesso, a falha e a duração de cada etapa.

Task	Função executada	Papel no fluxo
carregar_dim_produto	executar_dim_produto()	Atualiza incrementalmente a dimensão de produtos.
carregar_dim_territorio	executar_dim_territorio()	Atualiza incrementalmente a dimensão de territórios.
carregar_dim_tempo	executar_dim_tempo()	Gera e garante a presença do calendário no DW.
carregar_fato_venda	executar_fato_venda()	Processa a fato após a conclusão das dimensões.

3.1 Dependências entre as tarefas

```
produto = carregar_dim_produto()
territorio = carregar_dim_territorio()
tempo = carregar_dim_tempo()
fato = carregar_fato_venda()

[produto, territorio, tempo] >> fato
```

A dependência acima garante que carregar_fato_venda somente seja liberada quando as três dimensões terminarem com sucesso. As dimensões podem ser executadas em paralelo, reduzindo o tempo total do fluxo e preservando a integridade referencial da tabela fato.

carregar_dim_produto	→	**carregar_fato_venda**
carregar_dim_territorio	→	**carregar_fato_venda**
carregar_dim_tempo	→	**carregar_fato_venda**

4. Validações antes da execução

Antes de disparar a DAG pela interface, foram executadas validações de sintaxe, registro no catálogo do Airflow e importação das tarefas.

Validação	Resultado	Status
python -m py_compile /opt/airflow/dags/etl_adventureworks_dw.py	Sem erro de sintaxe	**OK**
airflow dags list	DAG etl_adventureworks_dw listada	**OK**
airflow dags list-import-errors	No data found	**OK**
airflow tasks list etl_adventureworks_dw	Quatro tasks reconhecidas	**OK**

4.1 Saúde do ambiente

Com o ambiente completo em execução, o Airflow apresentou os principais componentes em estado saudável: banco de metadados, agendador, disparador e processador de DAG. Também não havia DAGs falhadas ou em execução antes do primeiro disparo manual.

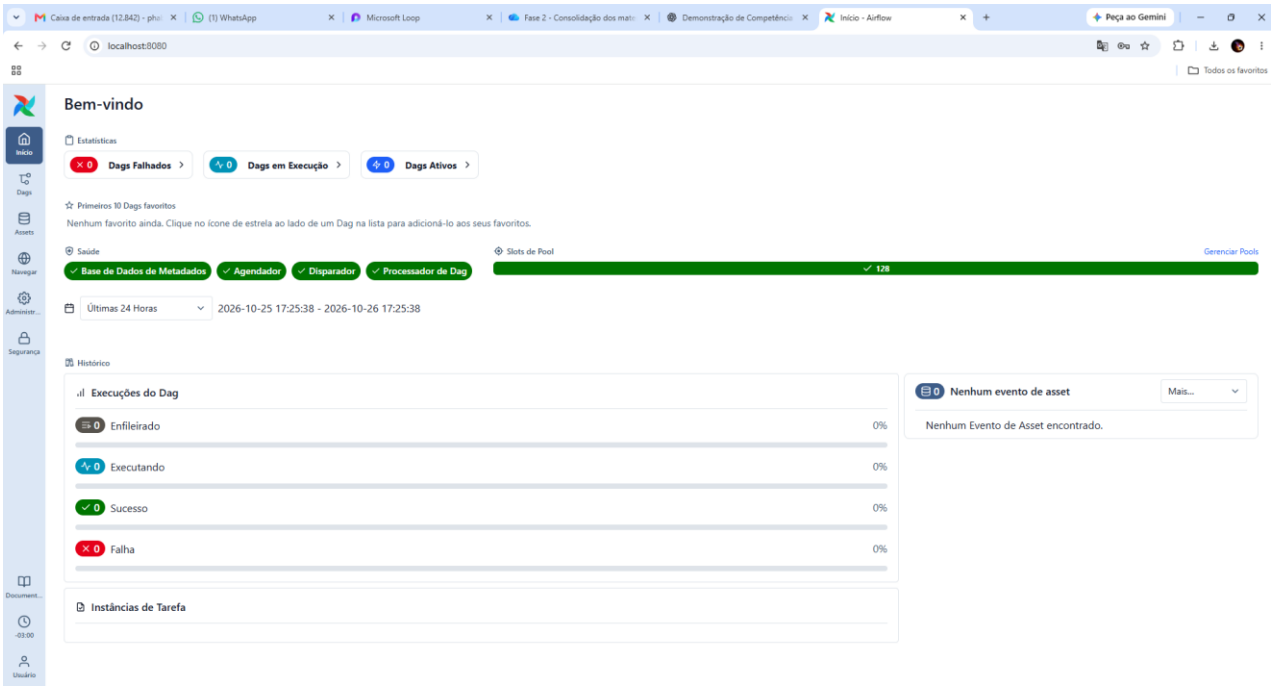


Figura 1 - Tela inicial do Airflow com os componentes principais em estado saudável.

4.2 Reconhecimento da DAG

A interface do Airflow reconheceu a DAG etl_adventureworks_dw e exibiu as quatro tarefas configuradas. A DAG permaneceu inicialmente pausada, conforme a configuração DAGS_ARE_PAUSED_AT_CREATION, até o acionamento manual para validação.

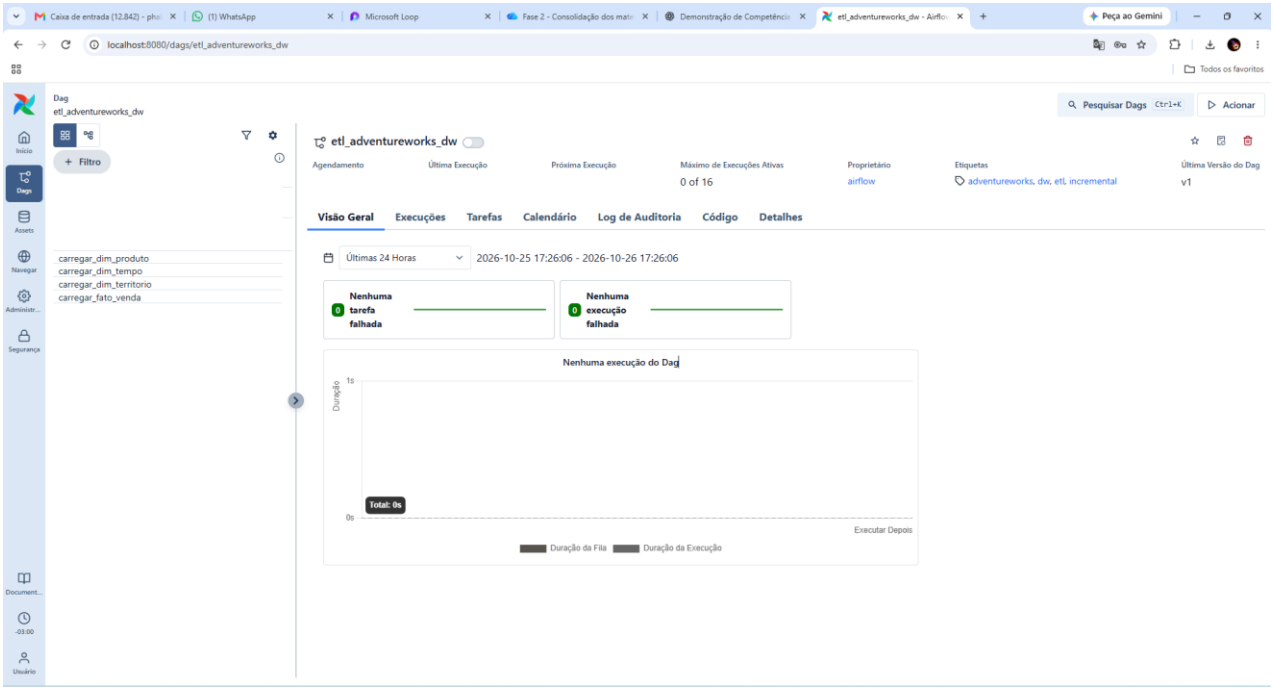


Figura 2 - DAG etl_adventureworks_dw reconhecida pela interface do Airflow, com as quatro tarefas cadastradas.

5. Execução manual da DAG

Após a validação da estrutura, a DAG foi ativada e acionada manualmente pela interface web do Airflow. A execução concluiu com estado global de sucesso, sem necessidade de novas tentativas em nenhuma task.

Task	Estado	Tentativas	Duração observada
carregar_dim_produto	**Sucesso**	1	00:00:11.205
carregar_dim_territorio	**Sucesso**	1	00:00:11.207
carregar_dim_tempo	**Sucesso**	1	00:00:01.749
carregar_fato_venda	**Sucesso**	1	00:05:05

Execução global: Sucesso | duração total aproximada de 00:05:21 | acionamento manual pelo usuário admin.

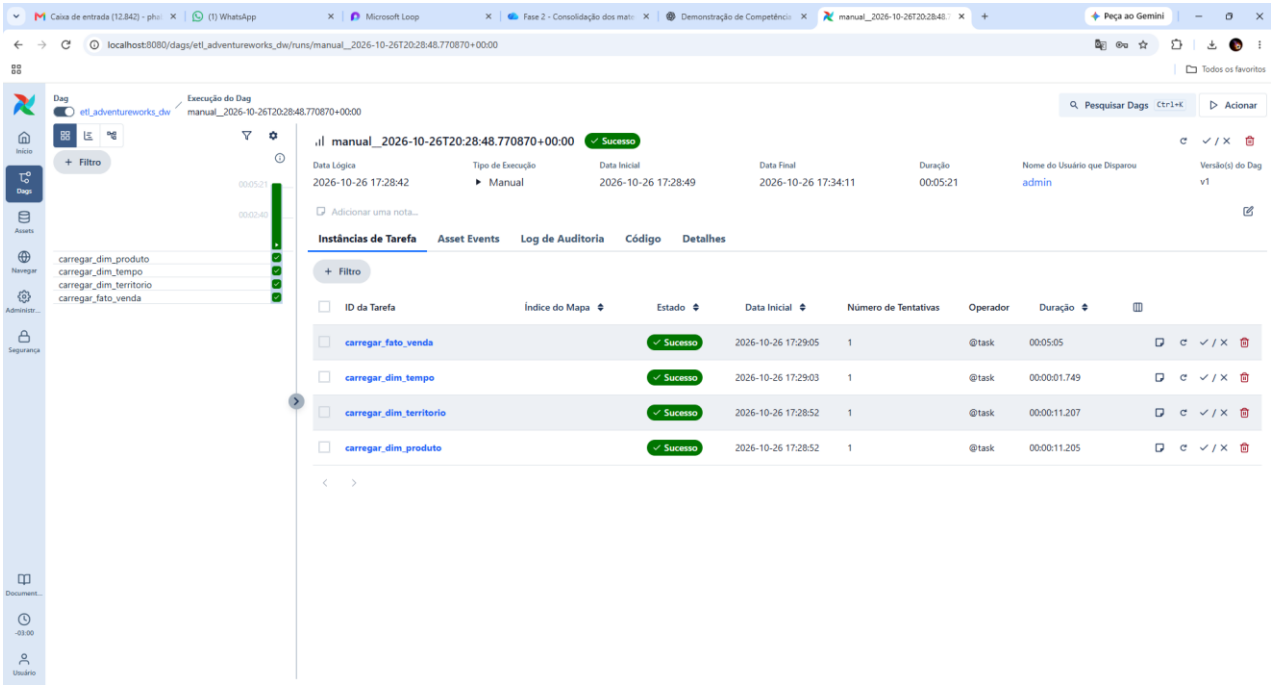


Figura 3 - Execução manual da DAG concluída com sucesso nas quatro tasks.

6. Validação pós-execução

Após a execução pelo Airflow, foi realizada uma conferência direta no PostgreSQL do Data Warehouse. As contagens permaneceram iguais às já validadas antes do disparo, demonstrando que a orquestração não gerou duplicidades nem alterou o volume esperado das tabelas.

Tabela	Quantidade de registros após a DAG
dw.dim_produto	505
dw.dim_territorio	11
dw.dim_tempo	1.127
dw.fato_venda	121.317

6.1 Watermarks após a execução

Os watermarks permaneceram nos maiores ModifiedDate já alcançados na origem. Isso indica que a execução do Airflow respeitou o estado incremental previamente estabelecido e não retrocedeu o controle de processamento.

Processo	ultima_execucao
dim_produto	2014-02-08 10:03:55.510
dim_territorio	2008-04-30 00:00:00
fato_venda	2014-07-07 00:00:00

Observação: dim_tempo não utiliza watermark. O calendário é reconstruído deterministicamente a partir do intervalo de datas da origem e a carga usa ON CONFLICT DO NOTHING para impedir duplicação.

7. Comportamento incremental observado

A execução pelo Airflow ocorreu depois de os watermarks já terem sido inicializados e validados em testes anteriores. Como não havia registros posteriores aos maiores ModifiedDate conhecidos, as contagens do DW e os watermarks permaneceram inalterados após a DAG. Esse comportamento é compatível com a estratégia incremental implementada: apenas registros com ModifiedDate maior que o watermark são elegíveis para extração em dim_produto, dim_territorio e fato_venda.

Mecanismo	Comportamento esperado	Evidência já validada
Watermark	Filtrar ModifiedDate > ultima_execucao	Reexecução sem mudanças retornou 0 para produto, território e fato.
Idempotência	Reprocessamento não duplica registros	505 produtos, 11 territórios, 1.127 datas e 121.317 itens mantidos.
Carga parcial	Processar apenas o subconjunto posterior	Teste controlado: 1 produto e 96 itens da fato foram processados.
Dependências	Fato somente após as dimensões	DAG executou as três dimensões antes de liberar fato_venda.

8. Observações técnicas

- A DAG foi mantida com schedule=None durante a validação para impedir execuções automáticas não controladas. Após a homologação, uma periodicidade pode ser definida conforme a necessidade do projeto.
- A tarefa carregar_dim_tempo informa processados=1127 com base na quantidade de linhas tentadas. Como a carga utiliza ON CONFLICT DO NOTHING, essa métrica representa tentativas de processamento e não necessariamente novos inserts.
- A execução de carregar_fato_venda foi a mais demorada, o que é coerente com o volume e com a consulta de custo histórico aplicada aos itens de venda.
- As credenciais permanecem fora do código-fonte e são fornecidas por variáveis de ambiente no arquivo .env, que não deve ser versionado.

9. Conclusão

A DAG etl_adventureworks_dw foi criada, importada e executada com sucesso no Apache Airflow. As quatro tasks foram reconhecidas sem erros de importação, a dependência entre dimensões e fato funcionou conforme projetado e a execução manual terminou integralmente em estado de sucesso. A validação pós-execução confirmou que o Data Warehouse manteve suas contagens esperadas e que os watermarks permaneceram consistentes. Dessa forma, a etapa de orquestração da ETL com Apache Airflow pode ser considerada concluída e documentada, fornecendo evidências suficientes para uso no relatório acadêmico e no artigo final do projeto.

Status final: ORQUESTRAÇÃO COM APACHE AIRFLOW VALIDADA ☑